Activity: Train CNN with Different Kernels

Introduction

In this activity, you'll explore the impact of **kernel size** on Convolutional Neural Networks (CNNs) using the **TensorFlow library**, a key aspect of hyperparameter tuning in deep learning for image classification tasks. You'll train CNN models with varying kernel sizes (e.g., 3x3, 5x5, 7x7) on the MNIST dataset, compare their training results (accuracy, loss, and computational cost), and analyze how kernel size affects feature extraction and model performance. This exercise builds on the concepts from the lesson on hyperparameter tuning, where kernel size was likened to a detective's magnifying glass, helping CNNs detect patterns like edges or shapes in images, and prepares you for optimizing CNN architectures for real-world applications.

Learning Outcomes

By the end of this activity, you will:

1. Load and preprocess the MNIST dataset for CNN training.
2. Build a CNN model with configurable kernel sizes using TensorFlow.
3. Train multiple CNN models with different kernel sizes and evaluate their performance.
4. Visualize training results (validation accuracy and loss) to compare the effects of kernel size.
5. Analyze how kernel size impacts feature extraction, model accuracy, and computational efficiency.

Working with Dev Container

To complete this assignment in a reliable and fully configured environment, please refer to the instructions in the file: `README-devcontainer.md`. This guide walks you through opening the assignment in **Visual Studio Code** using a **Dev Container**, which automatically installs all necessary Python libraries and tools. Following that setup ensures that the notebook runs smoothly without manual configuration or missing dependencies. Make sure to open the **main activity folder** in VS Code and follow the steps outlined in the Dev Container guide before starting the notebook.

Starter File

- Open the notebook: `activity-training-cnn-with-different-kernels.ipynb` in the `Code` folder.
- Run it locally or in Google Colab.
- Required libraries: `tensorflow`, `numpy`, and `matplotlib`.
- Use the MNIST dataset, which is automatically loaded via TensorFlow.

Activity Code Steps

This is a guided learning activity. You'll:

1. **Load and Preprocess the MNIST Dataset**
   Load the MNIST dataset (28x28 grayscale digit images) using TensorFlow, normalize pixel values to the

range [0,1], reshape the images for CNN input (e.g., adding a channel dimension), and convert labels to one-hot encoded format.

2. **Define a CNN Model with Configurable Kernel Size**
   Create a function to build a CNN model using TensorFlow, with two convolutional layers (32 and 64 filters), max pooling layers, and dense layers for classification, allowing the kernel size to be specified as a parameter for experimentation.
3. **Train Models with Different Kernel Sizes**
   Train three CNN models with kernel sizes 3x3, 5x5, and 7x7 for 5 epochs each, measuring training time and collecting performance metrics (validation accuracy, loss) for each model using the MNIST test set.
4. **Visualize and Compare Training Results**
   Generate plots comparing validation accuracy and loss across epochs for each kernel size, and print a summary of test accuracy, test loss, and training time to evaluate performance differences.
5. **Analyze Results**
   - Evaluate the accuracy and loss achieved by each kernel size on the test set.
   - Assess the computational cost (training time) as kernel size increases.
   - Consider how kernel size affects feature extraction (e.g., smaller kernels for fine details vs. larger kernels for broader patterns) and model performance on MNIST.

Conclusion

In this activity, you:

- Loaded and preprocessed the MNIST dataset for CNN training.
- Built a CNN model with configurable kernel sizes using TensorFlow.
- Trained multiple CNN models with kernel sizes 3x3, 5x5, and 7x7, evaluating their performance.
- Visualized validation accuracy and loss to compare the effects of kernel size.
- Analyzed how kernel size impacts feature extraction, model accuracy, and computational efficiency.

You've gained insights into how kernel size influences CNN performance, balancing feature extraction (e.g., fine details with 3x3 kernels vs. broader patterns with 7x7 kernels) with computational cost, as seen in the lesson on hyperparameter tuning. This understanding enhances your ability to optimize CNN architectures for tasks like image classification, building on concepts like layer depth and pooling, and prepares you for tackling more complex deep learning challenges.